

Intro to ML HW5

Dunkun Wei 110612025

December 4 2025

1 Problem and Data

1.1 Dataset choice

Problem: Image multi-class classification task (10 classes).

Dataset: Fashion-MNIST (28*28 grayscale images)

1.2 Class examples

Class: Different label is correspond to different class

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	shirt
7	Sneaker
8	Bag
9	Ankle boot

Preprocess input:

For NN model: Flatten the input from 28*28 into 784-D vector.

For CNN model: Add one dimension to change input from (batch size, height, width) $\rightarrow$ (batchsize, 1, height, width)

1.3 Split strategy

Use random split function to split train.csv into train-dataset and validation dataset, i choose the split percentage with 7 percent, means 93 percent train data and 7 percent validation data. We use seed to allow us reproduce the same scene.

2 Model

2.1 NN(Baseline and Improved)

2.1.1 NN Baseline

Architecture: Input $\rightarrow$ Linear $\rightarrow$ ReLU $\rightarrow$ Linear $\rightarrow$ ReLU $\rightarrow$ Linear $\rightarrow$ Softmax (in loss)

Split the whole structure into 3 full-connected network

INPUT SIZE(784) $\xrightarrow{fc_1}$ hidden dim1(512) $\xrightarrow{fc_2}$ hidden dim(256) $\xrightarrow{fc_1}$ NUM CLASSES(10)

Parameters count : 535,818

2.1.2 Improved NN

Add BN and Dropout based on NN baseline.

Architecture: Architecture: Linear $\rightarrow$ (BN) $\rightarrow$ ReLU $\rightarrow$ (Dropout) $\rightarrow$ Linear $\rightarrow$ (BN) $\rightarrow$ ReLU $\rightarrow$ (Dropout) $\rightarrow$ Linear $\rightarrow$ Softmax(inlossfunction)

Split the whole structure into 3 full-connected network

INPUT SIZE(784) $\xrightarrow{fc_1}$ hidden dim1(512) $\xrightarrow{fc_2}$ hidden dim(256) $\xrightarrow{fc_1}$ NUM CLASSES(10)

Parameters count : 537,354

2.2 CNN(Based and Improved)

2.2.1 CNN Baseline

Architecture: [Conv(3 $\times$ 3, 32)] $\rightarrow$ ReLU $\rightarrow$ [Conv(3 $\times$ 3, 64)] $\rightarrow$ ReLU $\rightarrow$ MaxPool(2 $\times$ 2) $\rightarrow$ Flatten $\rightarrow$ Linear $\rightarrow$ ReLU $\rightarrow$ Linear $\rightarrow$ Softmax (in loss)

Size change: Input size(batch size, 32, 28, 28) $\xrightarrow{conv1}$ (batchsize, 64, 28, 28) $\xrightarrow{conv2}$ (batchsize, 64, 14, 14) $\xrightarrow{flatten}$ (batchsize, 64*14*14) $\xrightarrow{fc_1}$ (batchsize, 128) $\xrightarrow{fc_2}$ (batchsize, 10)

Parameters setting stride = 1, padding = 1

Parameters count : 1,625,866

2.2.2 CNN improved

Add BN and Dropout and Residual block based on CNN baseline.

Architecture: x $\rightarrow$ Conv(1- > 16, 3*3) $\rightarrow$ ReLU $\rightarrow$ [Residual/ConvBlock1+Downsampling] $\rightarrow$ [Residual/ConvBlock2+Downsampling] $\rightarrow$ Flatten $\rightarrow$ Dropout $\rightarrow$ FC $\rightarrow$ Softmax(inloss)

Parameters setting : stride = 1, padding = 1

Parameters count : 103,802

3 Training

Training parameters setup:

Label	Class
epochs	15
batch _{size}	128
2	Pullover
lr	1e-3
Optimizer	Adam
Loss function	Cross entropy
val _{split}	0.07
device	cuda
dropout rate	0.25

Training environment: PyTorch 1.10.0+, GPU: Nvidia RTX 4050.

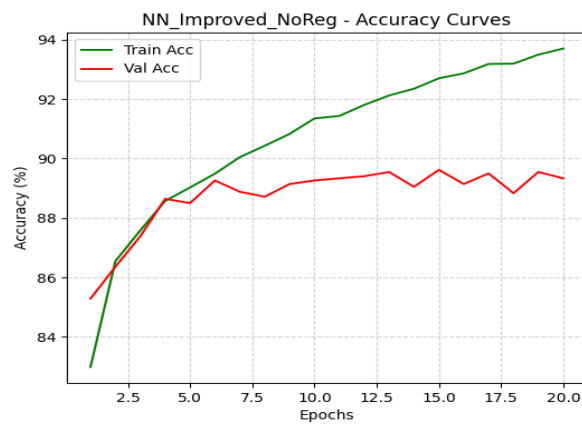
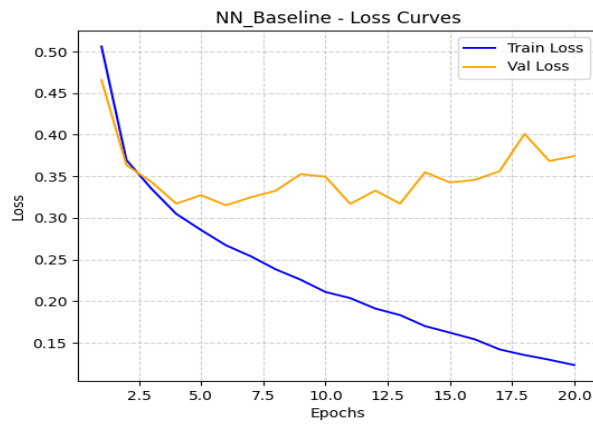
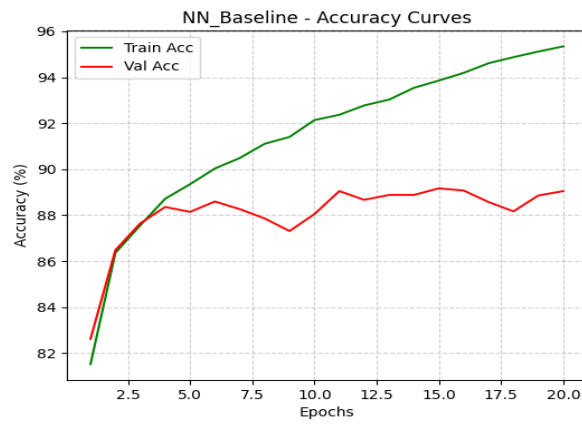
4 Results

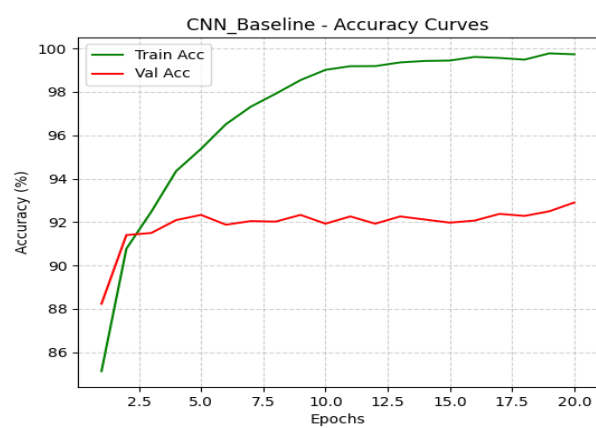
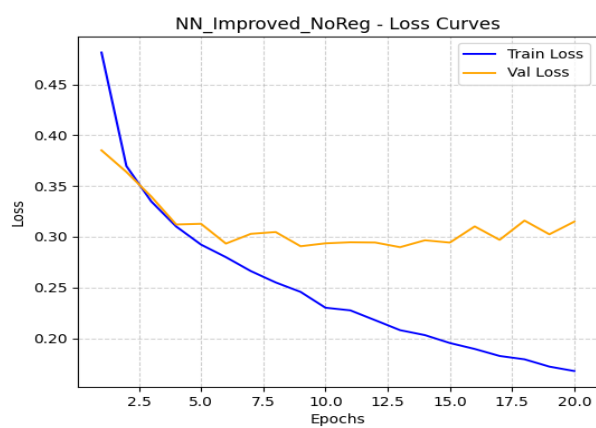
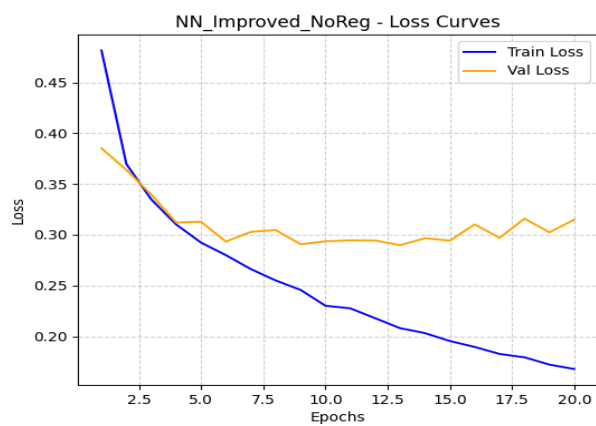
The performance table of different models:

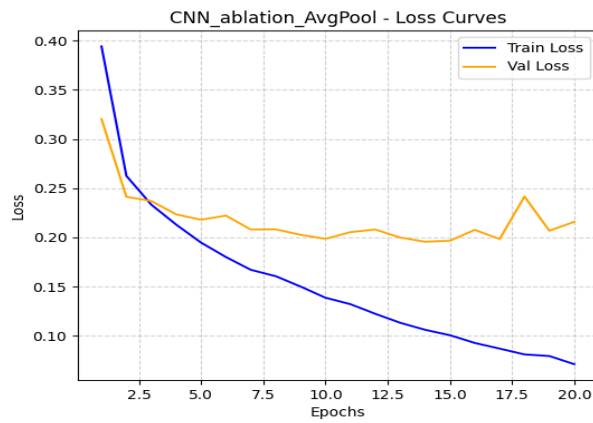
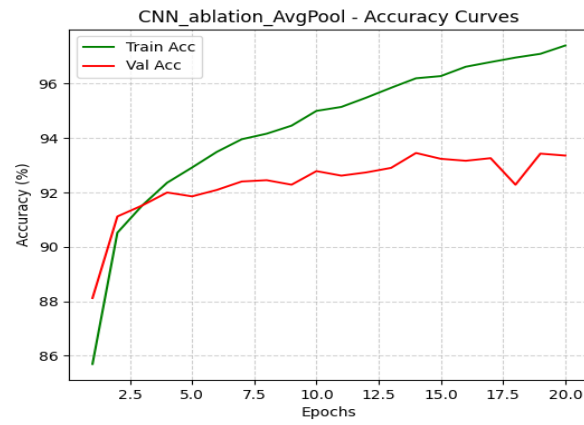
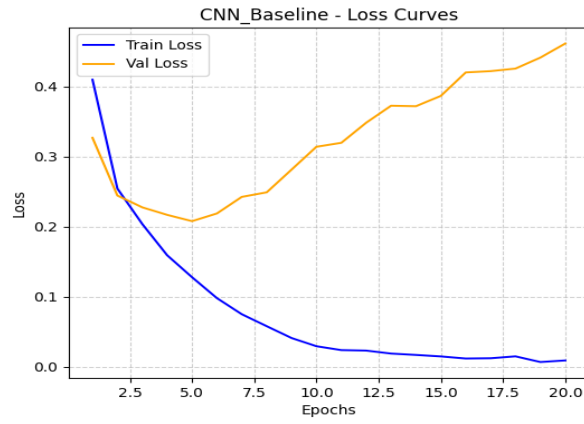
Table 1: Ablation Study Results Summary

Model Name	Model Type	Total Parameters	Best Val Acc(%)
NN-Baseline	NN	535,818	89.17
NN-Improved-NoReg	NN	537,354	89.62
NN-ablation-NoBN	NN	535,818	89.33
NN-ablation-NoDropout	NN	537,354	89.50
CNN-Baseline	CNN	1,625,866	92.90
CNN-Improved-Full	CNN	103,802	91.79
CNN-ablation-NoBN	CNN	103,194	92.17
CNN-ablation-NoResidual	CNN	54,778	92.55
CNN-ablation-NoDropout	CNN	103,802	91.43
CNN-ablation-MaxPool	CNN	103,802	93.19
CNN-ablation-AvgPool	CNN	103,802	93.45
CNN-ablation-NoDownsampling	CNN	574,202	90.71

4.1 Learning curve

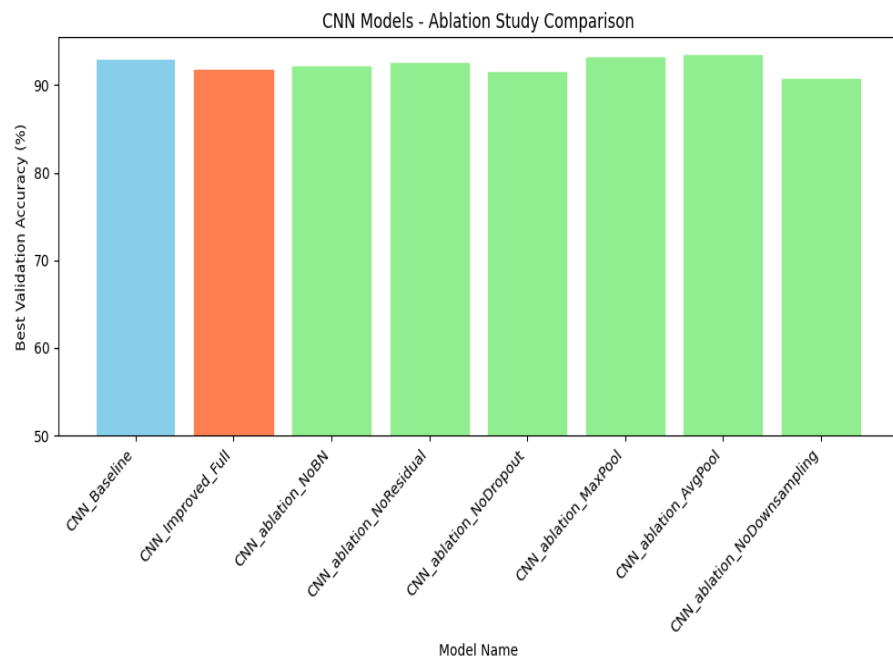
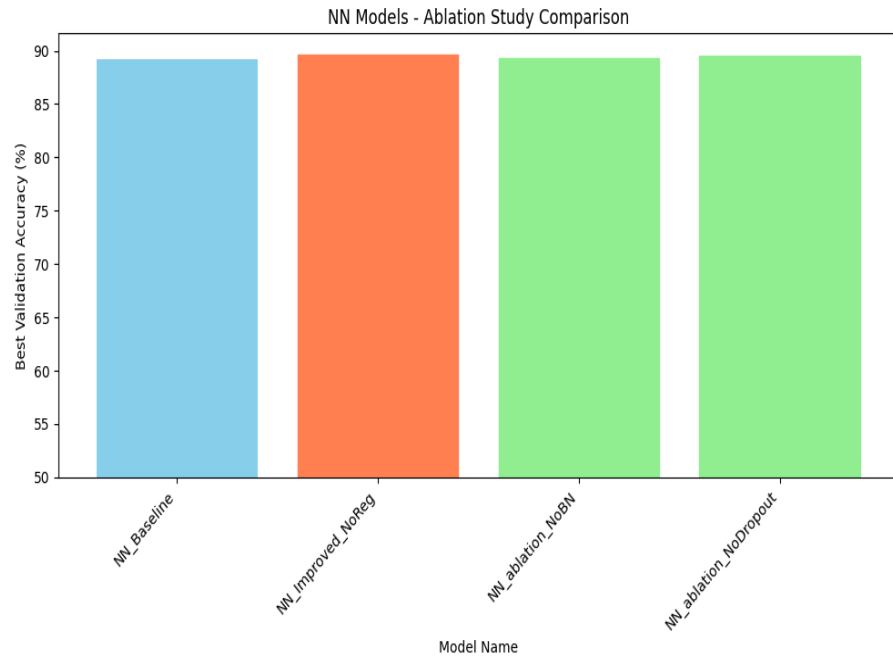






Result: According to the result table, the best performance model is CNN-ablation-AvgPool.

4.2 Ablation table



5 Discussion

1. NN vs CNN:

The most significant finding from the experiments is the performance and efficiency gap between the Fully Connected Network (NN) and the Convolutional Neural Network (CNN), even when trained on the same data and for the same task. NN's model have validation accuracy about 89 percent and about 500000 parameters, but CNN's model mostly have higher accuracy(92-93 percent) and less parameters.

When does CNN help and why? The CNN helps when processing structured data with spatial correlation, such as images, due to its Inductive Biases:

(1.) Locality (Local Connectivity): CNN filters only process small, local regions of the input image. This aligns with the natural structure of images where pixels close together are more related (e.g., forming edges or corners) than distant pixels. NN layers, being fully connected, treat all 784 input pixels equally, ignoring spatial relationships.

(2.) Weight Sharing (Translational Invariance): A CNN filter learns a single set of weights (a feature detector) and applies it across the entire image. This significantly reduces the total number of parameters compared to a fully connected layer, where every connection has a unique weight. This weight sharing makes the CNN translationally invariant—a learned feature can be recognized regardless of where it appears in the image.

2. Capacity and Regularization Trade-offs:

* NN Improved: Required high capacity (more parameters) and strong regularization (Dropout, BN) to generalize well. Without Dropout, it likely suffered from severe overfitting due as it easily memorizes the large number of weights.

* CNN Improved: By inherently building structure into the architecture (Weight Sharing), the model's effective capacity for image tasks is extremely high, despite its low parameter count. The architecture itself acts as a form of implicit regularization.

3. Role of BN, Dropout, Residual, and Pooling (Ablation Analysis)

(1.) Batch Normalization (BN)

* Observed Effect (Accuracy): BN typically yields a negative Accuracy (e.g., -0.3% to -0.5%)

* Principle: BN regularizes the model by standardizing the inputs to each layer, mitigating the problem of internal covariate shift. This allows the use of higher learning rates, making the training process robust and less sensitive to initialization.

(2.) Dropout

* Observed Effect (Accuracy): Dropout often results in a minor decrease in training accuracy but an increase in validation accuracy (e.g., +0.2% to

0.4%). In the NN, the effect might be less pronounced than in the CNN.

* Interpretation: Dropout acts as a powerful explicit regularization technique by randomly dropping neurons during training. This prevents co-adaptation of features and forces the network to learn more robust representations, thereby improving generalization.

(3.) Residual Connections (Skip Connections)

Theoratically, residual connection should provide providing a significant positive jump in performance (e.g., +0.4% to 0.5%) compared to an equivalent deep non-residual CNN.

* Interpretation: By adding the input x to the output of a block $F(x)$ (i.e., $H(x)=F(x)+x$), the network learns the residual mapping $F(x)$, which is generally easier than learning the complete transformation $H(x)$. Theoretically, this enables stable training of deeper networks by facilitating gradient flow through the identity shortcut, effectively addressing the vanishing gradient problem.

(4.) Pooling

* MaxPool/AvgPool vs Stride-2 Conv: In my case, AvgPooling have best accuracy than other two pooling method.

* No Downsampling: Led to a massive increase in parameters (e.g., from 100k to 500k) and either Out-of-Memory (OOM) errors or significantly worse performance. This decisively proves the necessity of downsampling in CNNs to reduce computational cost and increase the receptive field, thereby improving efficiency and generalization.